

```
In [1]: import pandas as pd
```

```
In [2]: df = pd.read_csv('customer_signups (1).csv')
df.shape
```

```
Out[2]: (515, 10)
```

```
In [3]: df.head()
```

```
Out[3]:
```

	customer_id	name	email	signup_date	source	region
0	CUST0340	Michael Johnson	nina75@example.com	2025-02-18	Instagram	Central
1	CUST0041	Ashley Johnson	matthew13@example.net	2025-01-16	Referral	North
2	CUST0187	Ana Good	rebecca44@example.net	2025-03-31	Instagram	NaN
3	CUST0480	Lucas Doyle	paularivera@example.org	2025-01-04	Google	Central
4	CUST0031	Shelley Chapman	whitneylawrence@example.org	2025-03-26	Instagram	North

◀  ▶

```
In [4]: df.isnull().sum()
```

```
Out[4]: customer_id      0
name                  0
email                26
signup_date          0
source               0
region              42
plan_selected        0
marketing_opt_in     0
age                 30
gender              22
dtype: int64
```

```
In [5]: df['gender'].unique()
```

```
Out[5]: array(['Female', 'Male', 'MALE', 'male', 'FEMALE', 'female', 'man',
              'Other', 'Woman', nan, 'M', 'NB', 'other', 'F', 'Non-binary'],
          dtype=object)
```

```
In [6]: df['plan_selected'].unique()
```

```
Out[6]: array(['Premium', 'Basic ', 'Pro', 'PRO', 'Basic', 'BASIC', 'Basci',
              'Premum', 'basic', 'Proe', 'PREMIUM', 'Pro ', 'premium',
              'Premium ', 'pro'], dtype=object)
```

```
In [7]: df['region'].unique()
```

```
Out[7]: array(['Central', 'North', nan, 'West', 'South', 'East'], dtype=object)
```

```
In [8]: df['source'].unique()
```

Out[8]: array(['Instagram', 'Referral', 'Google', 'Facebook', 'LinkedIn',
 'Direct'], dtype=object)

```
In [9]: df['age'].describe()
```

Out[9]: count 485.000000
 mean 33.451546
 std 9.591244
 min 0.000000
 25% 26.000000
 50% 33.000000
 75% 40.000000
 max 65.000000
 Name: age, dtype: float64

```
In [10]: df[df['age'] < 13]
```

Out[10]:

	customer_id	name	email	signup_date	source	region	pl
139	CUST0143	Anthony Morris	rossjennifer@example.org	2025-03-26	LinkedIn	South	
220	CUST0125	Michelle Simpson	michelle27@example.com	2025-03-28	Google	North	
267	CUST0449	Zachary Vasquez	taylor estrada@example.org	2025-01-16	Google	West	

```
In [11]: df['customer_id'].duplicated().sum()
```

Out[11]: np.int64(15)

```
In [12]: df[df.duplicated('customer_id', keep=False)]
```

Out[12]:

	customer_id	name	email	signup_date	source	region
15	CUST0007	Kelly Hill	NaN	2025-02-16	Google	East
40	CUST0137	Cindy Nelson	bburton@example.net	2025-01-19	Instagram	South
50	CUST0414	Tyrone Jackson	epetty@example.com	2025-02-21	Instagram	South
130	CUST0414	Tyrone Jackson	epetty@example.com	2025-02-21	Instagram	South
141	CUST0369	Scott Vaughan PhD	brownbrian@example.org	2025-01-02	Google	West
159	CUST0305	Gail Quinn	yjacobs@example.com	2025-01-18	Direct	West
170	CUST0493	Jennifer Wheeler	fordangela@example.org	2025-01-18	Google	North
171	CUST0356	Ashley Rodriguez	francojason@example.net	2025-01-25	Instagram	East
180	CUST0284	Hailey Smith	pricechristopher@example.net	2025-03-01	Google	South
185	CUST0305	Gail Quinn	yjacobs@example.com	2025-01-18	Direct	West
187	CUST0278	Brandon Adams	robert38@example.com	2025-03-23	Direct	South
195	CUST0301	Jose Arnold	martinpamela@example.org	2025-03-16	Referral	Central
210	CUST0025	Maureen Waller	sheltoncourtney@example.net	2025-01-22	Referral	North
219	CUST0449	Zachary Vasquez	taylorestrada@example.org	2025-01-16	Google	West
226	CUST0007	Kelly Hill	NaN	2025-02-16	Google	East
228	CUST0137	Cindy Nelson	bburton@example.net	2025-01-19	Instagram	South
254	CUST0248	Kelly Shah	jperry@example.org	2025-03-12	Instagram	North
256	CUST0098	Robert Montoya	greenjane@example.net	2025-02-16	Direct	North
258	CUST0068	Joseph Edwards	dtucker@example.net	2025-02-21	Google	East
267	CUST0449	Zachary Vasquez	taylorestrada@example.org	2025-01-16	Google	West
283	CUST0493	Jennifer Wheeler	fordangela@example.org	2025-01-18	Google	North

	customer_id	name	email	signup_date	source	region
321	CUST0369	Scott Vaughan PhD	brownbrian@example.org	2025-01-02	Google	West
336	CUST0025	Maureen Waller	sheltoncourtney@example.net	2025-01-22	Referral	North
341	CUST0068	Joseph Edwards	dtucker@example.net	2025-02-21	Google	East
358	CUST0098	Robert Montoya	greenjane@example.net	2025-02-16	Direct	North
383	CUST0301	Jose Arnold	martinpamela@example.org	2025-03-16	Referral	Central
450	CUST0248	Kelly Shah	jperry@example.org	2025-03-12	Instagram	North
467	CUST0356	Ashley Rodriguez	francojason@example.net	2025-01-25	Instagram	East
470	CUST0278	Brandon Adams	robert38@example.com	2025-03-23	Direct	South
480	CUST0284	Hailey Smith	pricechristopher@example.net	2025-03-01	Google	South

```
In [13]: dups = df[df.duplicated('customer_id', keep=False)].sort_values('customer_id')
dups
```

Out[13]:

	customer_id	name	email	signup_date	source	regio
15	CUST0007	Kelly Hill	NaN	2025-02-16	Google	Ea
226	CUST0007	Kelly Hill	NaN	2025-02-16	Google	Ea
336	CUST0025	Maureen Waller	sheltoncourtney@example.net	2025-01-22	Referral	Na
210	CUST0025	Maureen Waller	sheltoncourtney@example.net	2025-01-22	Referral	Na
341	CUST0068	Joseph Edwards	dtucker@example.net	2025-02-21	Google	Ea
258	CUST0068	Joseph Edwards	dtucker@example.net	2025-02-21	Google	Ea
358	CUST0098	Robert Montoya	greenjane@example.net	2025-02-16	Direct	Nort
256	CUST0098	Robert Montoya	greenjane@example.net	2025-02-16	Direct	Nort
228	CUST0137	Cindy Nelson	bburton@example.net	2025-01-19	Instagram	Sout
40	CUST0137	Cindy Nelson	bburton@example.net	2025-01-19	Instagram	Sout
450	CUST0248	Kelly Shah	jperry@example.org	2025-03-12	Instagram	Na
254	CUST0248	Kelly Shah	jperry@example.org	2025-03-12	Instagram	Na
187	CUST0278	Brandon Adams	robert38@example.com	2025-03-23	Direct	Sout
470	CUST0278	Brandon Adams	robert38@example.com	2025-03-23	Direct	Sout
480	CUST0284	Hailey Smith	pricechristopher@example.net	2025-03-01	Google	Sout
180	CUST0284	Hailey Smith	pricechristopher@example.net	2025-03-01	Google	Sout
195	CUST0301	Jose Arnold	martinpamela@example.org	2025-03-16	Referral	Centr
383	CUST0301	Jose Arnold	martinpamela@example.org	2025-03-16	Referral	Centr
185	CUST0305	Gail Quinn	yjacobs@example.com	2025-01-18	Direct	We
159	CUST0305	Gail Quinn	yjacobs@example.com	2025-01-18	Direct	We
171	CUST0356	Ashley Rodriguez	francojason@example.net	2025-01-25	Instagram	Ea

	customer_id	name	email	signup_date	source	region
467	CUST0356	Ashley Rodriguez	francojason@example.net	2025-01-25	Instagram	East
321	CUST0369	Scott Vaughan PhD	brownbrian@example.org	2025-01-02	Google	West
141	CUST0369	Scott Vaughan PhD	brownbrian@example.org	2025-01-02	Google	West
130	CUST0414	Tyrone Jackson	epetty@example.com	2025-02-21	Instagram	South
50	CUST0414	Tyrone Jackson	epetty@example.com	2025-02-21	Instagram	South
267	CUST0449	Zachary Vasquez	taylorestrada@example.org	2025-01-16	Google	West
219	CUST0449	Zachary Vasquez	taylorestrada@example.org	2025-01-16	Google	West
283	CUST0493	Jennifer Wheeler	fordangela@example.org	2025-01-18	Google	North
170	CUST0493	Jennifer Wheeler	fordangela@example.org	2025-01-18	Google	North

Removed 15 duplicate customer_id rows, keeping the most recent record for each customer.

```
In [14]: df = df.sort_values('signup_date')
df = df.drop_duplicates(subset='customer_id', keep='last')
```

```
In [15]: df[df['customer_id'] == 'CUST0449']
```

```
Out[15]:
```

	customer_id	name	email	signup_date	source	region	plan
219	CUST0449	Zachary Vasquez	taylorestrada@example.org	2025-01-16	Google	West	



```
In [16]: df.shape
```

```
Out[16]: (500, 10)
```

```
In [17]: df['customer_id'].duplicated().sum()
```

```
Out[17]: np.int64(0)
```

Standardized plan_selected and gender to fix casing and typos, using a dictionary of known variants mapped to clean categories.

```
In [18]: plan_map = {
    'basic': 'Basic',
    'basci': 'Basic',
    'pro': 'Pro',
    'proe': 'Pro',
    'premium': 'Premium',
    'premun': 'Premium',
}
```

```
In [19]: def clean_plan(val):
    key = str(val).strip().lower()
    return plan_map.get(key, str(val).strip().title())
```

```
In [20]: df['plan_selected'] = df['plan_selected'].apply(clean_plan)
```

```
In [21]: df['plan_selected'].unique()
```

```
Out[21]: array(['Basic', 'Premium', 'Pro'], dtype=object)
```

```
In [22]: gender_map = {
    'male': 'Male',
    'man': 'Male',
    'm': 'Male',
    'female': 'Female',
    'woman': 'Female',
    'f': 'Female',
    'nb': 'Other',
    'non-binary': 'Other',
}
```

```
In [23]: def clean_gender(val):
    if pd.isna(val):
        return np.nan
    key = str(val).strip().lower()
    return gender_map.get(key, str(val).strip().title())
```

```
In [24]: import numpy as np
```

```
In [25]: df['gender'] = df['gender'].apply(clean_gender)
df['gender'].unique()
```

```
Out[25]: array(['Female', 'Other', 'Male', nan], dtype=object)
```

Missing region (42 rows, 8.2%) flagged as 'Unknown' rather than guessed, since region can't be reliably inferred from other columns.

```
In [26]: df['region'] = df['region'].fillna('Unknown')
```

```
In [27]: df['region'].unique()
```

```
Out[27]: array(['Unknown', 'Central', 'East', 'North', 'South', 'West'],
    dtype=object)
```

```
In [28]: df['region'].isnull().sum()
```

```
Out[28]: np.int64(0)
```

Ages under 13 (2 rows) treated as data-entry errors and nulled out before imputing. All missing ages (32 total) filled with the median (33.5), which is robust to outliers

```
In [29]: df.loc[df['age'] < 13, 'age'] = np.nan
```

```
In [30]: df['age'].isnull().sum()
```

```
Out[30]: np.int64(32)
```

```
In [31]: age_median = df['age'].median()  
df['age'] = df['age'].fillna(age_median)
```

```
In [32]: df['age'].isnull().sum()
```

```
Out[32]: np.int64(0)
```

```
In [33]: df['age'].describe()
```

```
Out[33]: count    500.00000  
mean      33.65800  
std       9.06031  
min       18.00000  
25%      27.00000  
50%      33.50000  
75%      39.25000  
max       65.00000  
Name: age, dtype: float64
```

Missing or malformed emails (26 rows) flagged as invalid rather than guessed — these customers are unreachable by email until corrected at the source.

```
In [34]: import re  
  
email_pattern = re.compile(r'^^[^\s]+@[^\s]+\.[^\s]+$')
```

```
In [35]: def is_valid_email(val):  
    if pd.isna(val):  
        return False  
    return bool(email_pattern.match(str(val)))
```

```
In [36]: df['email_valid'] = df['email'].apply(is_valid_email)
```

```
In [37]: df[df['email_valid'] == False]
```

Out[37]:

	customer_id	name	email	signup_date	source	region	plan_selected	m
329	CUST0022	John Rich	NaN	2025-01-01	LinkedIn	East	Premium	
317	CUST0208	Jesse Huff	NaN	2025-01-03	Instagram	Unknown	Pro	
239	CUST0111	Jorge Hanson	NaN	2025-01-07	Google	North	Basic	
318	CUST0118	Jennifer Ellis	NaN	2025-01-23	Referral	North	Basic	
203	CUST0234	Mariah Meyer	NaN	2025-01-25	LinkedIn	Unknown	Basic	
11	CUST0236	Scott Pratt	NaN	2025-02-02	Google	East	Pro	
51	CUST0360	Jonathan Harris	NaN	2025-02-06	Google	South	Pro	
381	CUST0321	Micheal Brown	NaN	2025-02-06	Google	Central	Basic	
404	CUST0348	Emily Ray	not-an-email	2025-02-07	LinkedIn	North	Pro	
426	CUST0123	Dr. Ryan Dawson	NaN	2025-02-08	Instagram	South	Premium	
20	CUST0397	William Moore	NaN	2025-02-10	Facebook	North	Basic	
184	CUST0013	Stephanie Hoffman	NaN	2025-02-10	Direct	East	Pro	
226	CUST0007	Kelly Hill	NaN	2025-02-16	Google	East	Premium	
193	CUST0005	Amy Ellis	NaN	2025-02-18	LinkedIn	North	Basic	
118	CUST0081	Veronica Roberson	NaN	2025-02-23	Google	North	Basic	
227	CUST0151	Deanna Garza	NaN	2025-02-25	Google	South	Basic	
510	CUST0233	Robert Wilson	NaN	2025-02-27	Instagram	South	Pro	
511	CUST0202	David Rivers	NaN	2025-03-07	Instagram	South	Pro	
478	CUST0230	Mary Wells	NaN	2025-03-09	Instagram	East	Basic	
338	CUST0199	Anthony Frey	NaN	2025-03-09	Facebook	North	Basic	
483	CUST0080	Lisa Shaw	NaN	2025-03-11	Google	South	Pro	
181	CUST0378	Joel Gilbert	NaN	2025-03-14	Direct	West	Premium	

	customer_id	name	email	signup_date	source	region	plan_selected	m
310	CUST0287	Barbara Hester	NaN	2025-03-20	Google	Central	Pro	
369	CUST0160	Melissa Knight	NaN	2025-03-26	Referral	West	Basic	
376	CUST0419	Jerry Taylor	NaN	2025-03-28	Instagram	North	Basic	
351	CUST0095	Cody Thomas	NaN	2025-03-30	LinkedIn	North	Premium	

```
In [38]: df.loc[df['email_valid'] == False, 'email'] = np.nan
```

```
In [39]: df['email'].isnull().sum()
```

```
Out[39]: np.int64(26)
```

```
In [40]: df.isnull().sum()
```

```
Out[40]: customer_id      0
name                  0
email                26
signup_date          0
source               0
region              0
plan_selected        0
marketing_opt_in     0
age                 0
gender              20
email_valid          0
dtype: int64
```

```
In [41]: df = df.drop(columns=['email_valid'])
```

```
In [42]: df['gender'] = df['gender'].fillna('Unknown')
```

```
In [43]: df['gender'].unique()
```

```
Out[43]: array(['Female', 'Other', 'Male', 'Unknown'], dtype=object)
```

```
In [44]: df['gender'].isnull().sum()
```

```
Out[44]: np.int64(0)
```

```
In [45]: df.to_csv('customer_signups_clean.csv', index=False)
```

```
In [46]: df['source'].value_counts()
```

```
Out[46]: source
        Google      173
        Instagram  106
        Referral    89
        LinkedIn    58
        Direct      43
        Facebook    31
        Name: count, dtype: int64
```

```
In [47]: df['plan_selected'].value_counts()
```

```
Out[47]: plan_selected
        Basic      190
        Pro        179
        Premium    131
        Name: count, dtype: int64
```

```
In [48]: df['region'].value_counts()
```

```
Out[48]: region
        North      123
        South      102
        West        88
        East        82
        Central     65
        Unknown     40
        Name: count, dtype: int64
```

```
In [49]: pd.crosstab(df['gender'], df['marketing_opt_in'])
```

```
Out[49]: marketing_opt_in  No  Yes
```

gender		
Female	94	128
Male	95	124
Other	16	23
Unknown	9	11

```
In [50]: pd.crosstab(df['gender'], df['marketing_opt_in'], normalize='index').mul(100).round(1)
```

```
Out[50]: marketing_opt_in  No  Yes
```

gender		
Female	42.3	57.7
Male	43.4	56.6
Other	41.0	59.0
Unknown	45.0	55.0

```
In [51]: df.dtypes
```

```
Out[51]: customer_id      object
        name              object
        email             object
        signup_date       object
        source            object
        region            object
        plan_selected     object
        marketing_opt_in  object
        age               float64
        gender            object
        dtype: object
```

```
In [52]: df['signup_date'] = pd.to_datetime(df['signup_date'])
```

```
In [53]: df.dtypes
```

```
Out[53]: customer_id      object
        name              object
        email             object
        signup_date       datetime64[ns]
        source            object
        region            object
        plan_selected     object
        marketing_opt_in  object
        age               float64
        gender            object
        dtype: object
```

```
In [54]: weekly = df.set_index('signup_date').resample('W')['customer_id'].count()
        weekly
```

```
Out[54]: signup_date
2025-01-05    27
2025-01-12    40
2025-01-19    50
2025-01-26    31
2025-02-02    33
..
2026-07-19     0
2026-07-26     0
2026-08-02     0
2026-08-09     0
2026-08-16     2
Freq: W-SUN, Name: customer_id, Length: 85, dtype: int64
```

```
In [55]: df[df['signup_date'] > '2025-04-01']
```

Out[55]:

	customer_id	name	email	signup_date	source	region
387	CUST0035	Casey McLaughlin	jonathanmartin@example.net	2026-08-15	Google	Central
502	CUST0292	Chad Mcdowell	uking@example.com	2026-08-15	Direct	West



2 rows had a signup_date 18 months outside the rest of the dataset (Aug 2026 vs Jan-Mar 2025) — flagged and excluded only from the time-based trend chart.

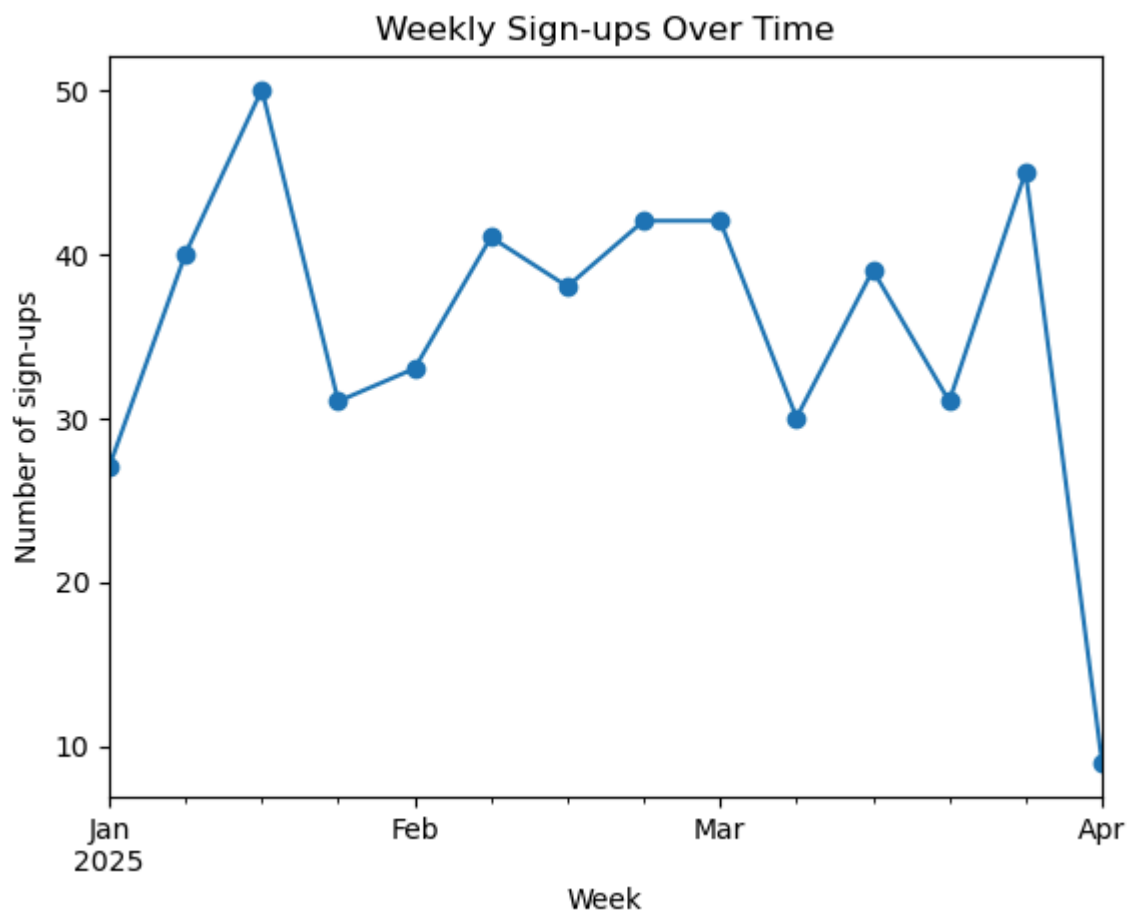
```
In [56]: df['implausible_date'] = df['signup_date'] > '2025-04-01'
```

```
In [57]: df_for_trend = df[df['implausible_date'] == False]
weekly = df_for_trend.set_index('signup_date').resample('W')['customer_id'].count
weekly
```

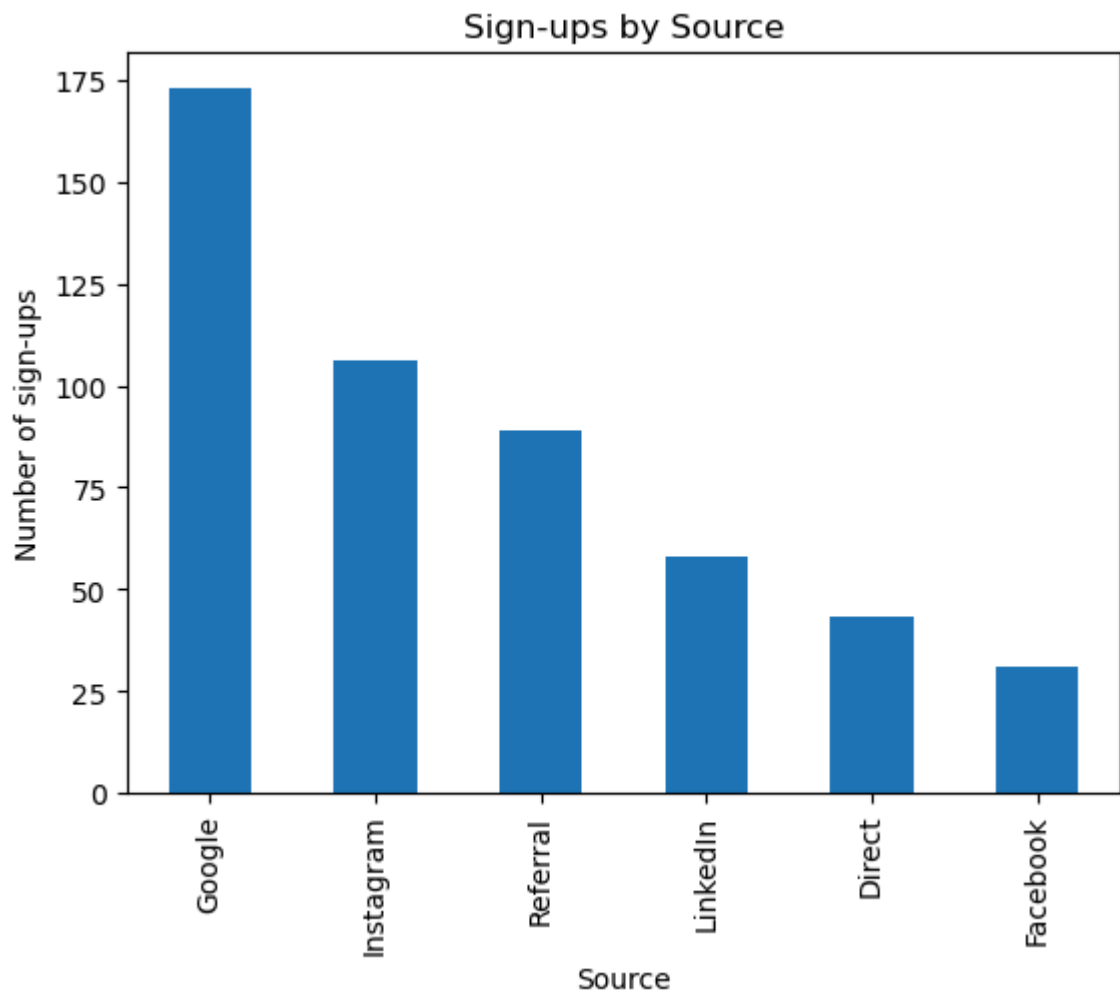
```
Out[57]: signup_date
2025-01-05    27
2025-01-12    40
2025-01-19    50
2025-01-26    31
2025-02-02    33
2025-02-09    41
2025-02-16    38
2025-02-23    42
2025-03-02    42
2025-03-09    30
2025-03-16    39
2025-03-23    31
2025-03-30    45
2025-04-06     9
Freq: W-SUN, Name: customer_id, dtype: int64
```

```
In [58]: import matplotlib.pyplot as plt

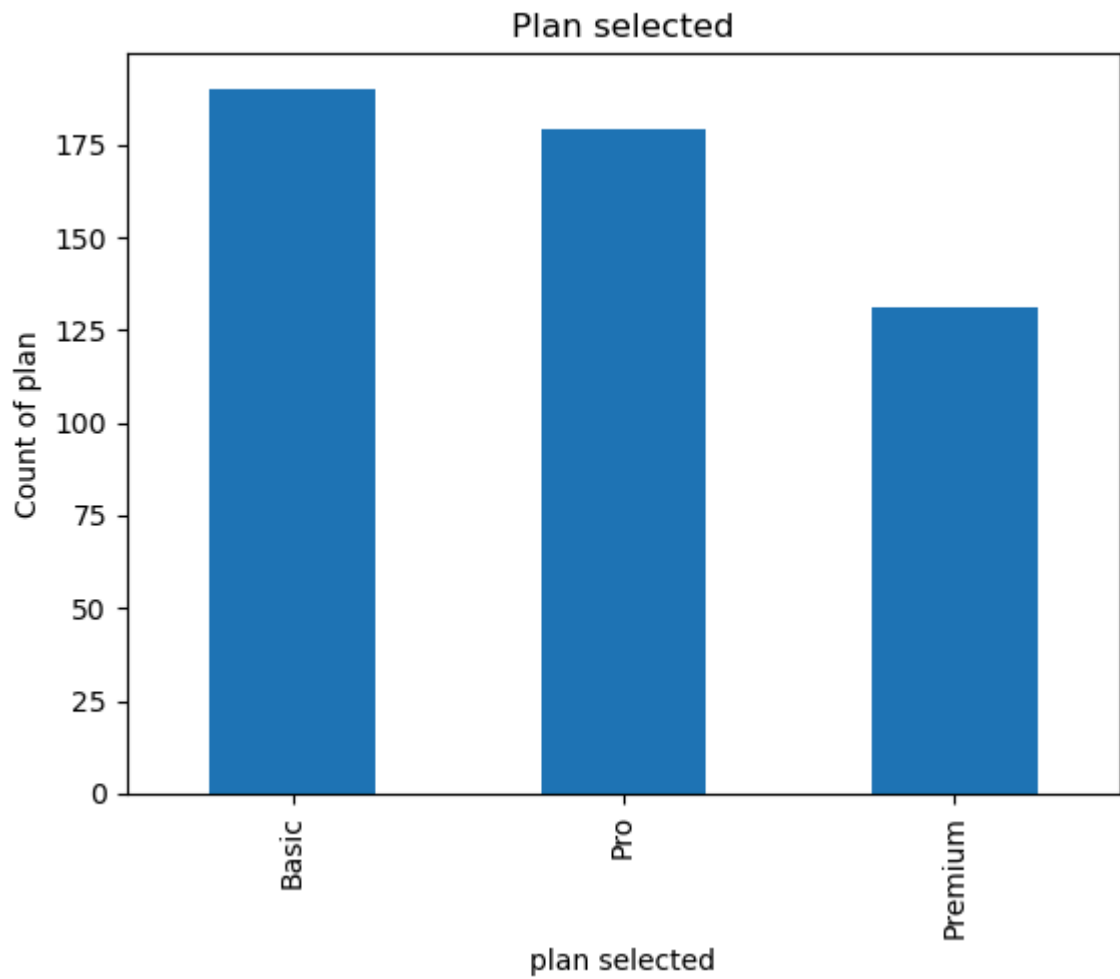
weekly.plot(kind='line', marker='o', title='Weekly Sign-ups Over Time')
plt.xlabel('Week')
plt.ylabel('Number of sign-ups')
plt.show()
```



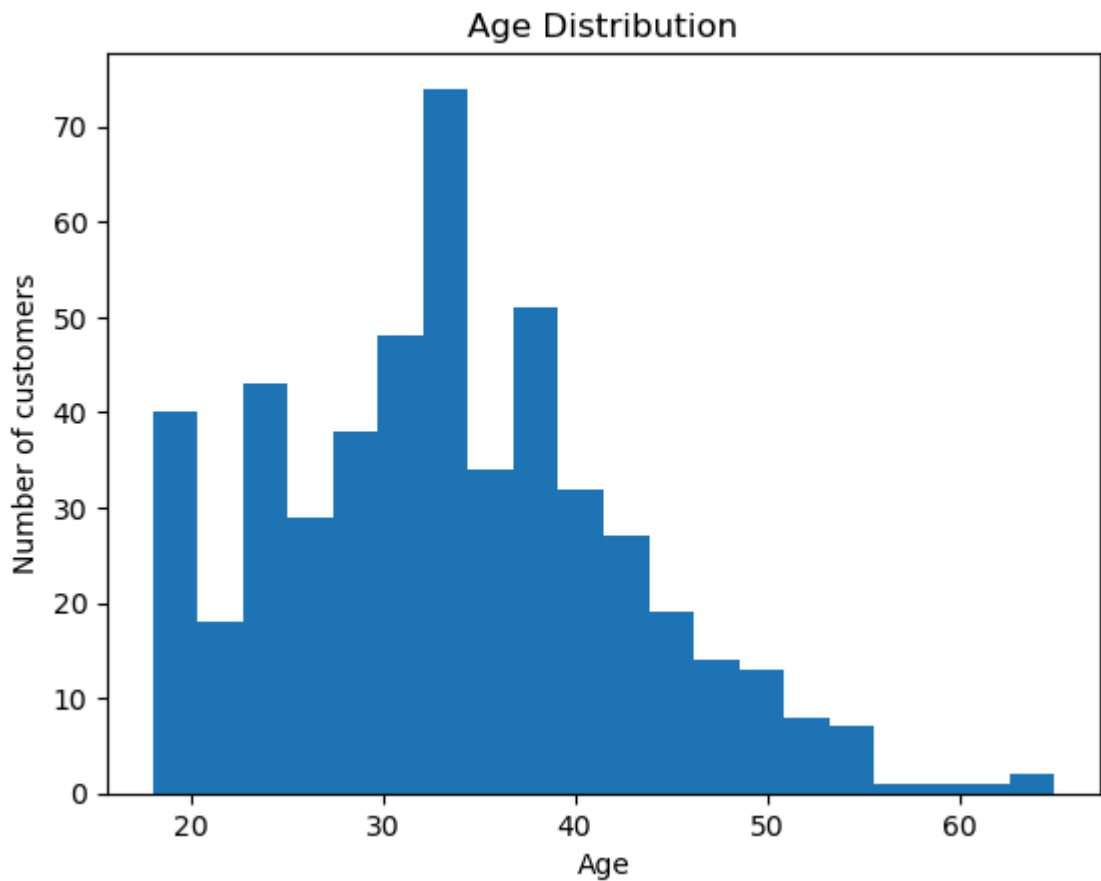
```
In [59]: df['source'].value_counts().plot(kind='bar', title='Sign-ups by Source')
plt.xlabel('Source')
plt.ylabel('Number of sign-ups')
plt.show()
```



```
In [60]: df['plan_selected'].value_counts().plot(kind='bar', title='Plan selected')
plt.xlabel('plan selected')
plt.ylabel('Count of plan')
plt.show()
```



```
In [61]: df['age'].plot(kind='hist', bins=20, title='Age Distribution')
plt.xlabel('Age')
plt.ylabel('Number of customers')
plt.show()
```



```
In [62]: tickets = pd.read_csv('support_tickets.csv')
tickets.head()
```

```
Out[62]:
```

	ticket_id	customer_id	ticket_date	category	status	resolution_days
0	TKT00141	CUST0483	2025-01-07	Account	Resolved	0.0
1	TKT00081	CUST0181	2025-01-15	Account	Pending	NaN
2	TKT00085	CUST9994	2025-03-15	Technical	Open	6.0
3	TKT00198	CUST0183	2025-04-18	Billing	Open	0.0
4	TKT00004	CUST0187	2025-04-12	Other	Open	NaN

```
In [63]: tickets.isnull().sum()
```

```
Out[63]: ticket_id      0
customer_id    0
ticket_date    0
category       0
status         0
resolution_days 47
dtype: int64
```

```
In [64]: merged = tickets.merge(df[['customer_id', 'signup_date', 'plan_selected']], on='
merged.head()
```

```
Out[64]:
```

	ticket_id	customer_id	ticket_date	category	status	resolution_days	signup_date
0	TKT00141	CUST0483	2025-01-07	Account	Resolved	0.0	2025-01-06
1	TKT00081	CUST0181	2025-01-15	Account	Pending	NaN	2025-01-07
2	TKT00198	CUST0183	2025-04-18	Billing	Open	0.0	2025-03-26
3	TKT00004	CUST0187	2025-04-12	Other	Open	NaN	2025-03-31
4	TKT00086	CUST0343	2025-03-29	Billing	Open	1.0	2025-03-28

```
In [65]: merged['ticket_date'] = pd.to_datetime(merged['ticket_date'])
merged['signup_date'] = pd.to_datetime(merged['signup_date'])
```

```
In [66]: merged['days_to_ticket'] = (merged['ticket_date'] - merged['signup_date']).dt.days
merged[['customer_id', 'signup_date', 'ticket_date', 'days_to_ticket']].head()
```

```
Out[66]:
```

	customer_id	signup_date	ticket_date	days_to_ticket
0	CUST0483	2025-01-06	2025-01-07	1
1	CUST0181	2025-01-07	2025-01-15	8
2	CUST0183	2025-03-26	2025-04-18	23
3	CUST0187	2025-03-31	2025-04-12	12
4	CUST0343	2025-03-28	2025-03-29	1

```
In [67]: merged['early_contact'] = merged['days_to_ticket'] <= 14
```

```
In [68]: early_customers = merged[merged['early_contact'] == True]
early_by_plan = early_customers.drop_duplicates('customer_id')['plan_selected'].
early_by_plan
```

```
Out[68]: plan_selected
Pro      52
Premium  36
Basic    30
Name: count, dtype: int64
```

```
In [69]: total_by_plan = df['plan_selected'].value_counts()
early_pct_by_plan = (early_by_plan / total_by_plan * 100).round(1)
early_pct_by_plan
```

```
Out[69]: plan_selected
Basic    15.8
Premium  27.5
Pro      29.1
Name: count, dtype: float64
```

```
In [70]: march = df[(df['signup_date'] >= '2025-03-01') & (df['signup_date'] <= '2025-03-31')]
march['source'].value_counts()
```

```
Out[70]: source
Google      49
Instagram   38
Referral    31
LinkedIn    22
Facebook    14
Direct      14
Name: count, dtype: int64
```

```
In [71]: df['age_group'] = pd.cut(df['age'], bins=[17,25,35,45,55,66], labels=['18-25','26-35','36-45','46-55','56-65'])
```

```
In [72]: pd.crosstab(df['age_group'], df['marketing_opt_in'], normalize='index').mul(100)
```

```
Out[72]: marketing_opt_in  No  Yes
age_group
18-25      38.6  61.4
26-35      40.6  59.4
36-45      45.9  54.1
46-55      52.2  47.8
56-65      40.0  60.0
```

```
In [73]: pd.crosstab(df['age_group'], df['plan_selected'])
```

```
Out[73]: plan_selected  Basic  Premium  Pro
age_group
18-25      43      25   33
26-35      72      55   75
36-45      55      34   57
46-55      18      15   13
56-65       2       2    1
```

```
In [74]: march = df[(df['signup_date'] >= '2025-03-01') & (df['signup_date'] <= '2025-03-31')]
march['source'].value_counts()
```

```
Out[74]: source
Google      49
Instagram   38
Referral    31
LinkedIn    22
Facebook    14
Direct      14
Name: count, dtype: int64
```

```
In [75]: audit_data = {
    'Issue': ['Duplicate customer_id', 'Missing region', 'Inconsistent plan_selected', 'Missing age', 'Missing/malformed email', 'Inconsistent gender', 'Missing gender', 'Implausible age (<13)', 'Out-of-range signup_date'],
    'Count': ['15 rows', '42 rows (8.2%)', '12 raw values → 3', '30 rows (5.8%)']
}
```

```

        '26 rows (5.0%)', '11 raw values → 3', '22 rows (4.3%)', '2 rows',
        'Severity': ['High', 'High', 'High', 'Medium', 'Medium', 'Medium', 'Low', 'L
    }
    audit_table = pd.DataFrame(audit_data)
    audit_table

```

Out[75]:

	Issue	Count	Severity
0	Duplicate customer_id	15 rows	High
1	Missing region	42 rows (8.2%)	High
2	Inconsistent plan_selected	12 raw values → 3	High
3	Missing age	30 rows (5.8%)	Medium
4	Missing/malformed email	26 rows (5.0%)	Medium
5	Inconsistent gender	11 raw values → 3	Medium
6	Missing gender	22 rows (4.3%)	Low
7	Implausible age (<13)	2 rows	Low
8	Out-of-range signup_date	2 rows	Low

In [76]:

```

region_missing_pct = 42 / 515 * 100
print(f"Region itself is missing for {region_missing_pct:.1f}% of raw rows (42 o

```

Region itself is missing for 8.2% of raw rows (42 of 515) – the largest single data gap in the dataset.

In []:

```


```